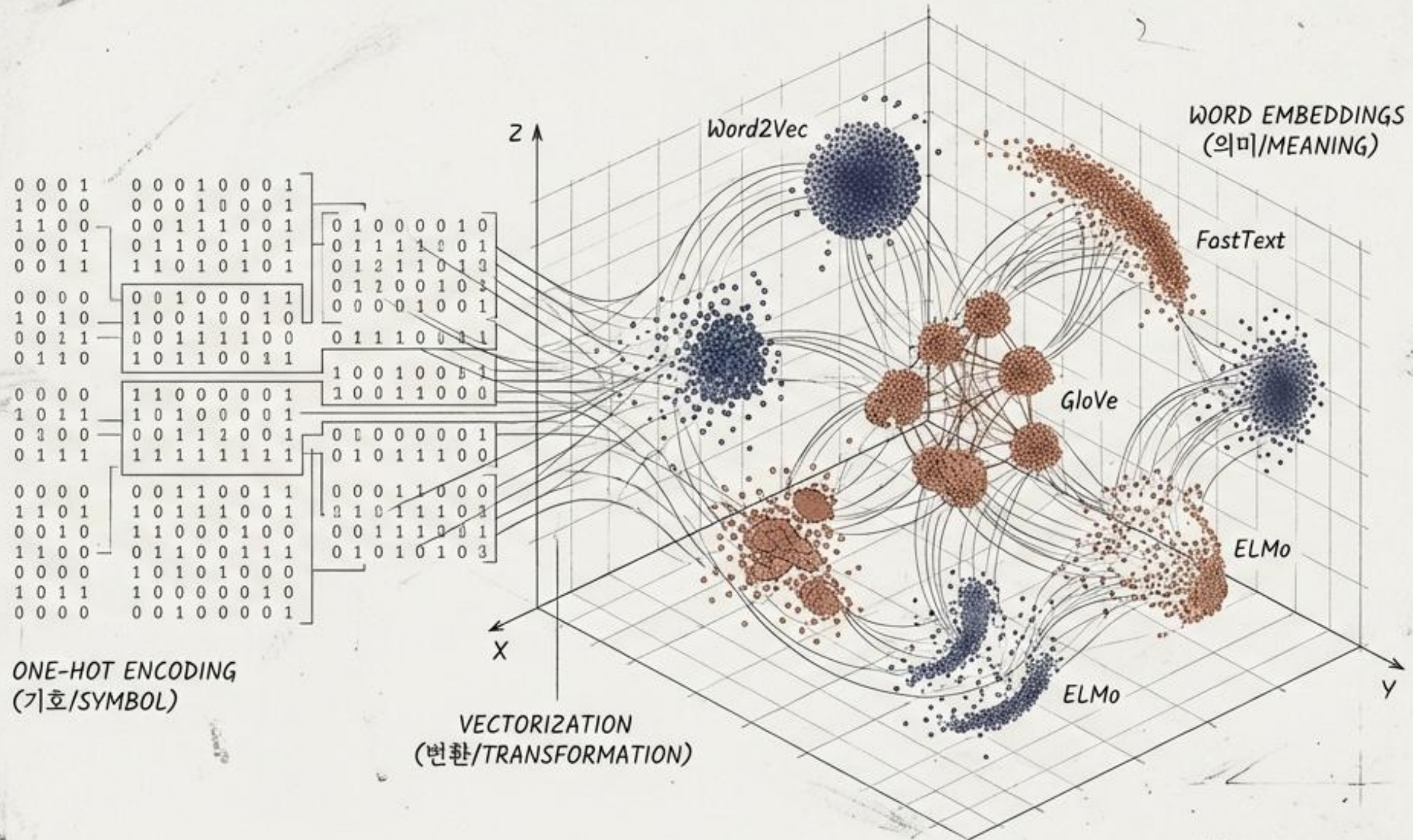


기호에서 의미로: 다차원 공간의 탐험

자연어 처리의 단어 임베딩과 벡터화 기법

원-핫 인코딩부터 Word2Vec, GloVe, FastText, 그리고 ELMo까지



기계의 언어 이해 첫 단추: 단어 집합과 정수 인코딩

단어 집합(Vocabulary): 중복을 배제한 텍스트 내 고유 단어들의 집합

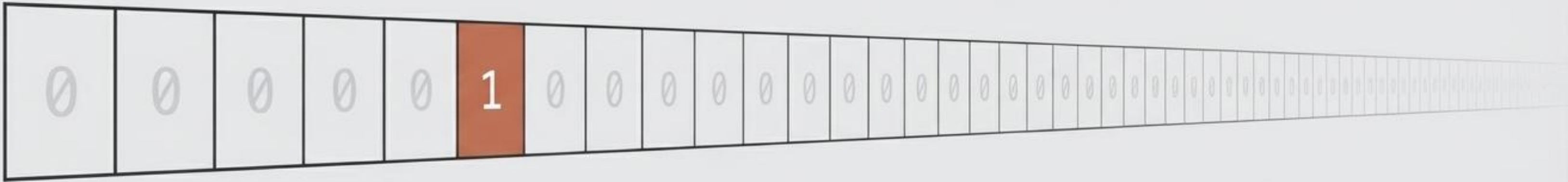
정수 인코딩: 텍스트를 기계가 처리할 수 있도록 각 단어에 고유한 인덱스 부여



```
from konlpy.tag import Okt
okt = Okt()
token = okt.morphs('나는 자연어 처리를 배운다')
word2index = {'나': 0, '는': 1, '자연어': 2, '처리': 3, '를': 4, '배운다': 5}
```

고립된 차원의 한계: 원-핫 인코딩(One-hot Encoding)

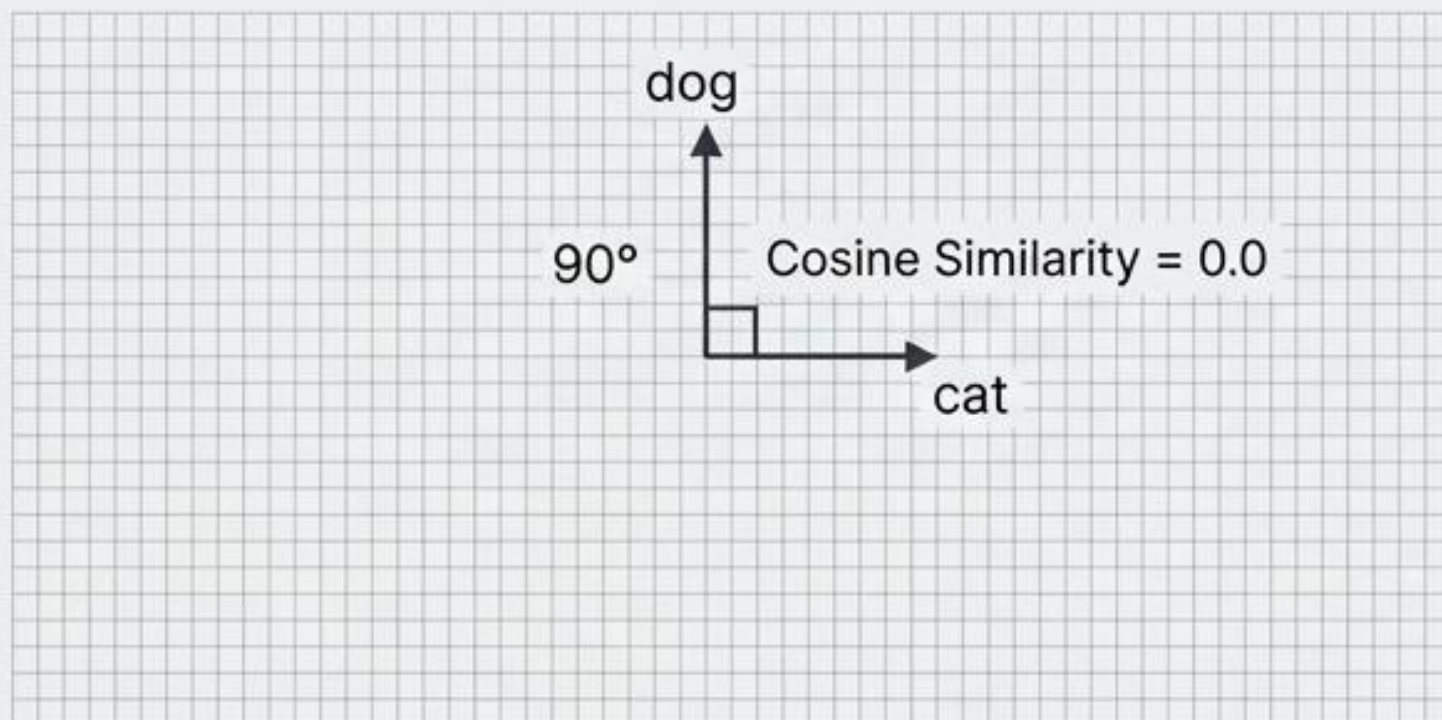
- 희소표현(Sparse Representation): 단어 집합 크기를 차원으로 삼고, 해당 인덱스만 1, 나머지는 0으로 표현
- 공간적 한계: 10,000개의 단어면 10,000차원. 극심한 저장 공간 낭비
- 의미적 한계: 단어 간의 '유사도'를 전혀 표현할 수 없음



```
def one_hot_encoding(word, word2index):  
    one_hot_vector = [0]*(len(word2index))  
    index = word2index[word]  
    one_hot_vector[index] = 1  
    return one_hot_vector  
one_hot_encoding('자연어', word2index)  
Output: [0, 0, 1, 0, 0, 0]
```


유사도 '0'의 절망, 밀집 벡터(Dense Vector)로의 진화

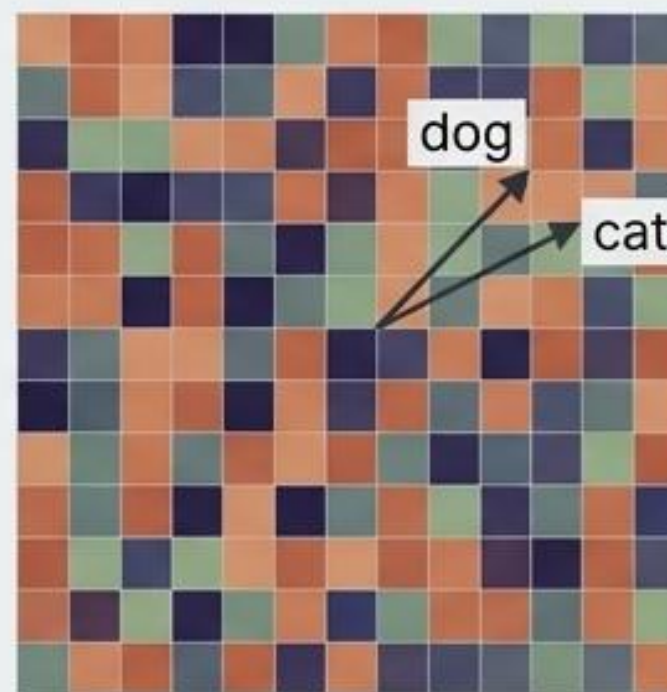
희소 벡터 (Sparse)



원-핫 벡터 간 내적은 항상 직교(Orthogonal)하여 유사도 계산 불가

```
dog = torch.FloatTensor([1, 0, 0, 0, 0])
cat = torch.FloatTensor([0, 1, 0, 0, 0])
print(torch.cosine_similarity(dog, cat, dim=0))
Output: tensor(0.)
```

밀집 벡터 (Dense)

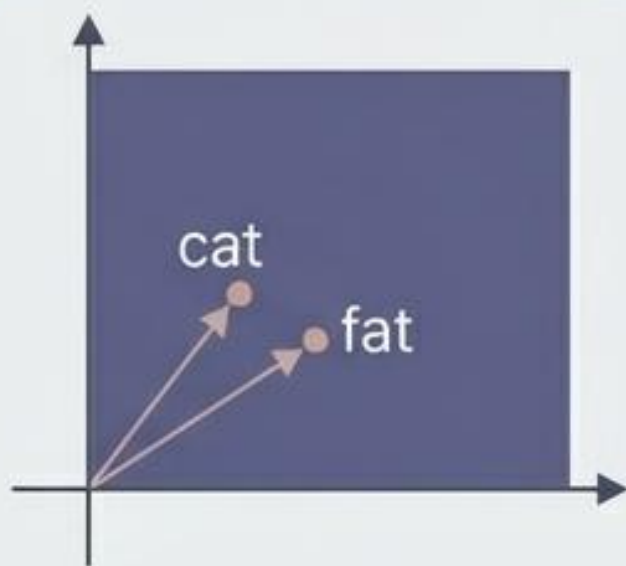
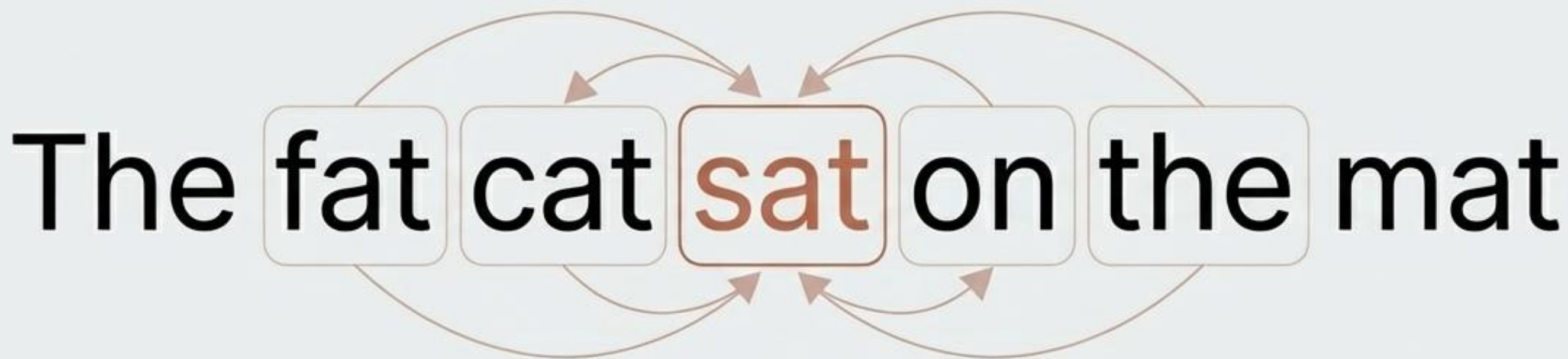


사용자가 설정한 저차원(예 : 128)의 실수값으로 압축하는 임베딩 벡터 도입

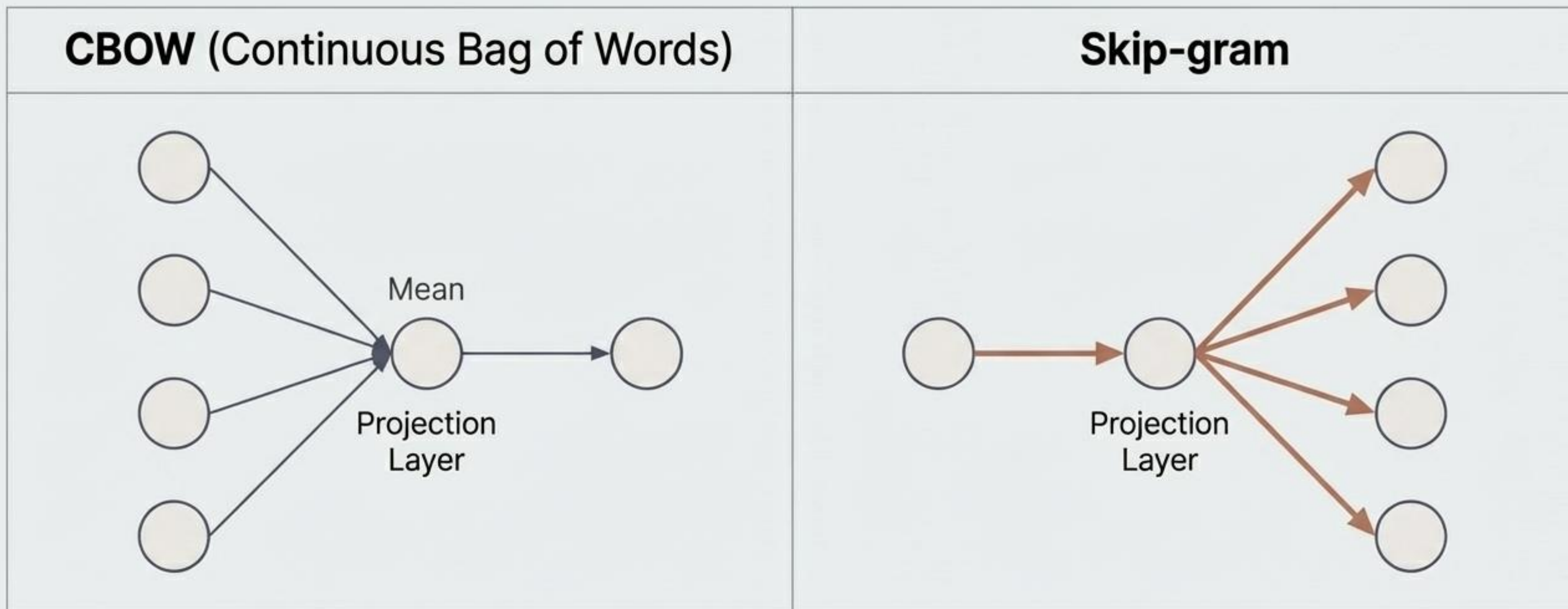
```
dog = torch.FloatTensor([1, 0, 0, 0, 0])
cat = torch.FloatTensor([0, 1, 0, 0, 0])
print(torch.cosine_similarity(dog, cat, dim=0))
Output: tensor(0.)
```

주변이 곧 의미를 만든다: 분산 표현(Distributed Representation)

- 분포 가설(Distributional Hypothesis): 비슷한 위치에 등장하는 단어들은 비슷한 의미를 가진다
- 벡터 연산 증명: 한국 - 서울 + 도쿄 = 일본



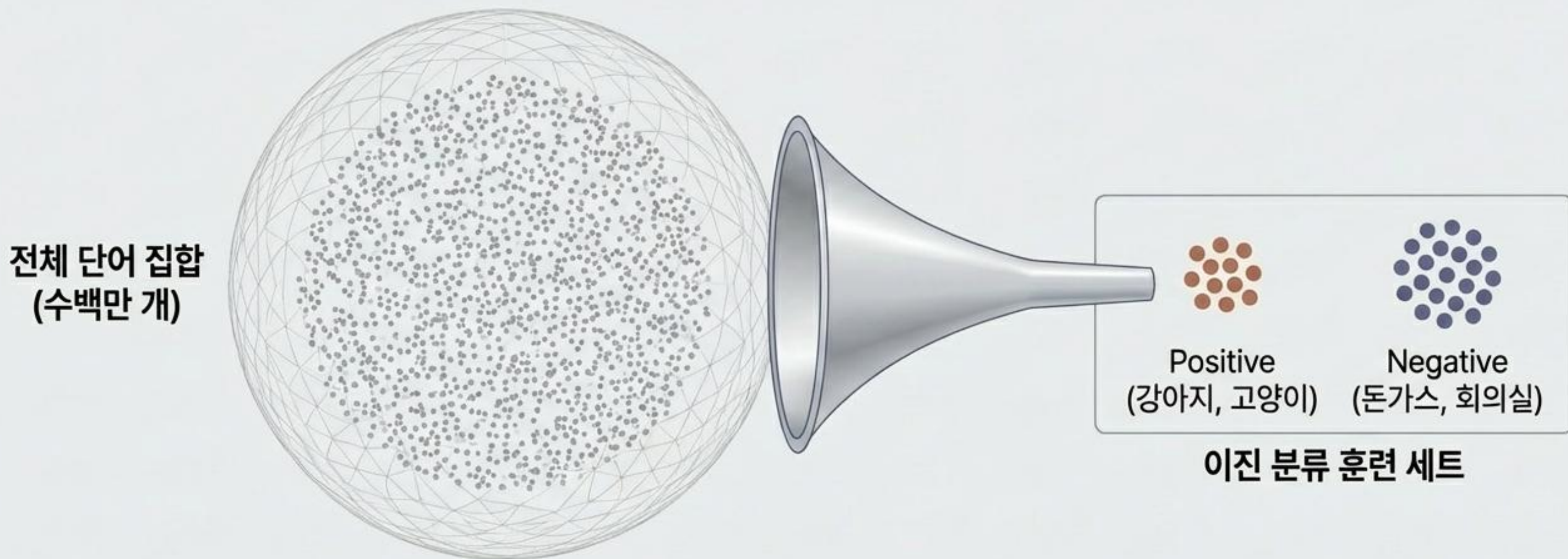
Word2Vec 아키텍처: CBOW vs Skip-gram



- **얇은 신경망(Shallow NN):** 활성화 함수가 없는 단 1개의 투사층(Projection Layer)
- 일반적으로 예측 문맥이 넓은 Skip-gram 구조가 성능 우위를 가짐

연산의 한계를 돌파하다: 네거티브 샘플링(Negative Sampling)

- 다중 클래스 분류 문제를 이진 분류(Binary Classification)로 변환
- 수백만 단어 전체의 Softmax 연산을 배제하고 속도 극대화



Word2Vec 훈련 실습 및 단어 유사도 검증

ENG 모델 (TED 데이터)

```
model = Word2Vec(sentences=result,  
                 vector_size=100,  
                 window=5, min_count=5,  
                 sg=0)  
  
model.wv.most_similar('man')  
Output: [('woman', 0.842), ('guy', 0.817),  
         ('boy', 0.777)]
```

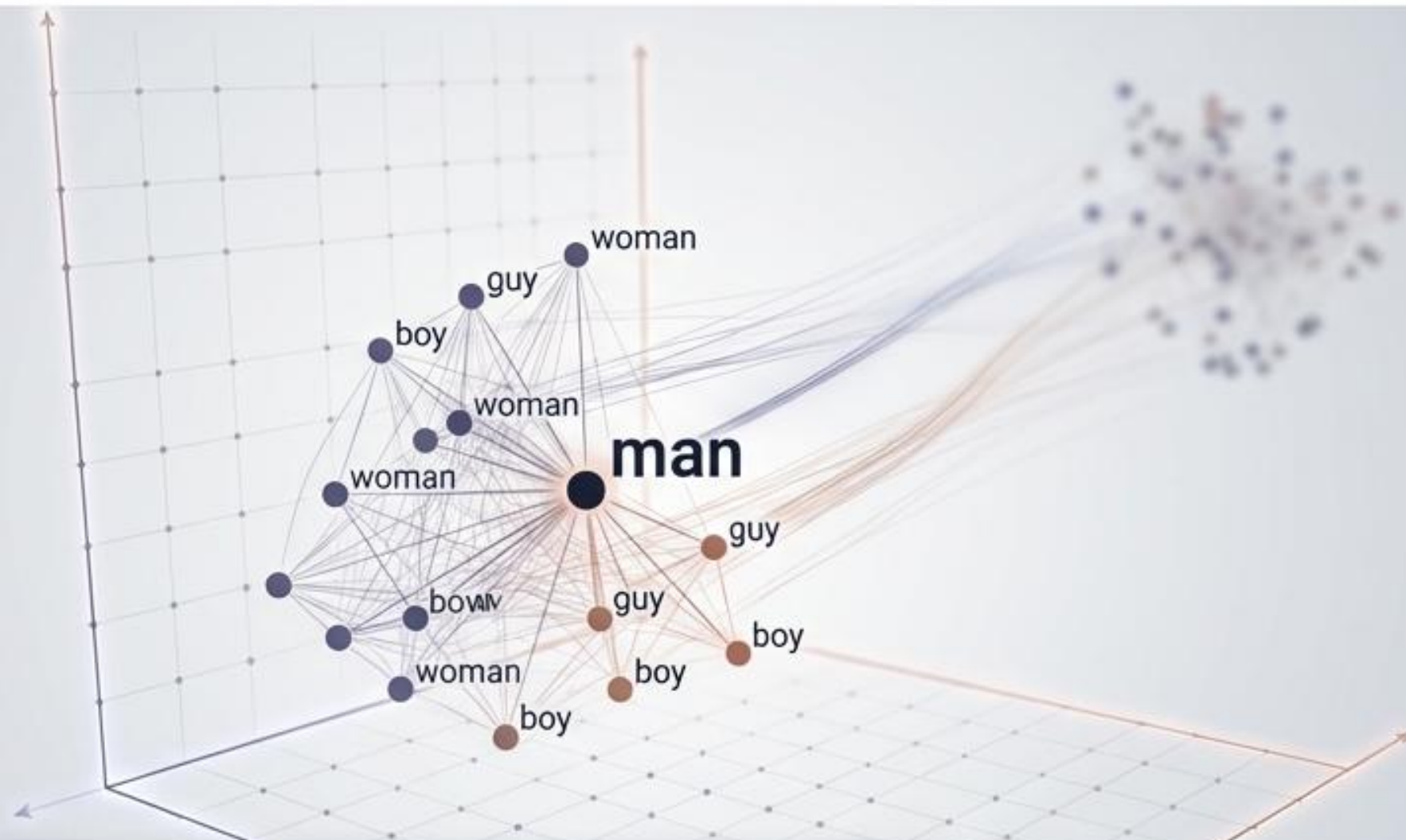
KOR 모델 (네이버 영화 리뷰)

```
model.wv.vectors.shape  
Output: (16477, 100)  
  
model.wv.most_similar('최민식')  
Output: [('한석규', 0.878), ('안성기',  
                             0.875), ('김수현', 0.855)]  
  
model.wv.most_similar('히어로')  
Output: [('슬래셔', 0.874), ('느와르', 0.866),  
         ('무협', 0.842)]
```

방대한 코퍼스를 바탕으로 고차원 벡터의 의미적 유사성을 실수값으로 도출

다차원 공간의 시각화: Embedding Projector

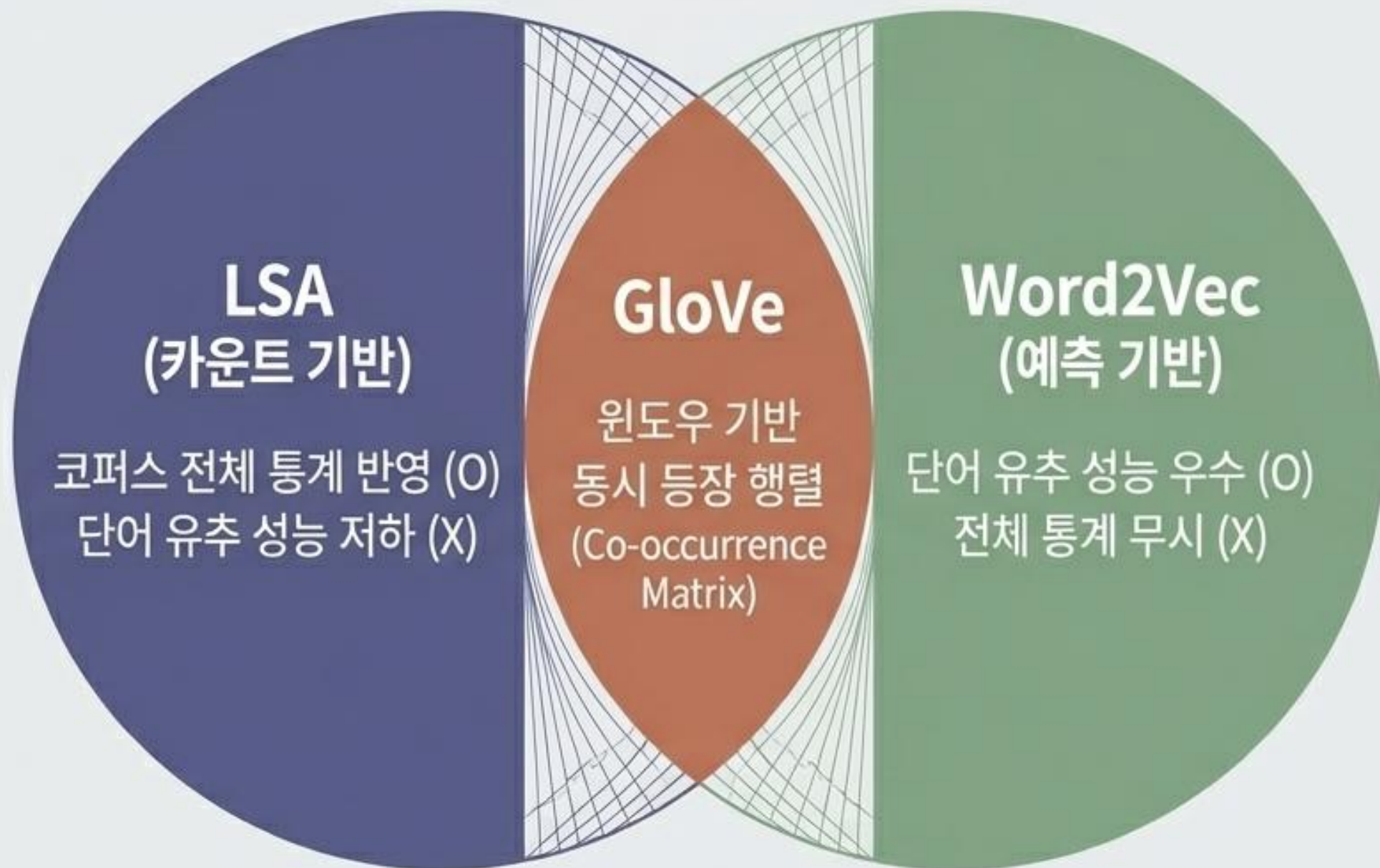
PCA 및 t-SNE 알고리즘을 통한 고차원 벡터의 차원 축소 및 군집화



bash

```
!python -m gensim.scripts.word2vec2tensor --input eng_w2v --output eng_w2v  
생성 파일: eng_w2v_metadata.tsv, eng_w2v_tensor.tsv
```

통계와 예측의 융합: GloVe (Global Vectors)의 철학



임베딩된 두 단어 벡터의 내적이 전체 코퍼스에서의 동시 등장 확률이 되도록 학습

GloVe의 핵심: 동시 등장 확률의 크기 관계 비(Ratio)

확률 및 비율	k = solid	k = gas	k = water	k = fashion
$P(k \mid \text{ice})$	0.00019	0.0000066	0.003	0.0000017
$P(k \mid \text{steam})$	0.0000022	0.000078	0.0022	0.0000018
Ratio: $P(k \text{ice}) / P(k \text{steam})$	8.9	0.085	1.36	0.96

- k=solid: ice와 밀접하므로 비율이 1보다 큼 (8.9)
- k=gas: steam과 밀접하므로 비율이 1보다 훨씬 작음 (0.085)
- k=water / fashion: 양쪽 모두와 연관되거나 무관하여 1에 수렴

GloVe의 손실 함수(Loss Function) 도출 설계도

비율 정보를 벡터 공간에 인코딩

$$F(w_i - w_j, w_k) = \frac{P_{ik}}{P_{jk}}$$

준동형(Homomorphism) 만족 및 지수 함수 도입

$$w_i^T w_k = \log(P_{ik}) = \log(X_{ik}) - \log(X_i)$$

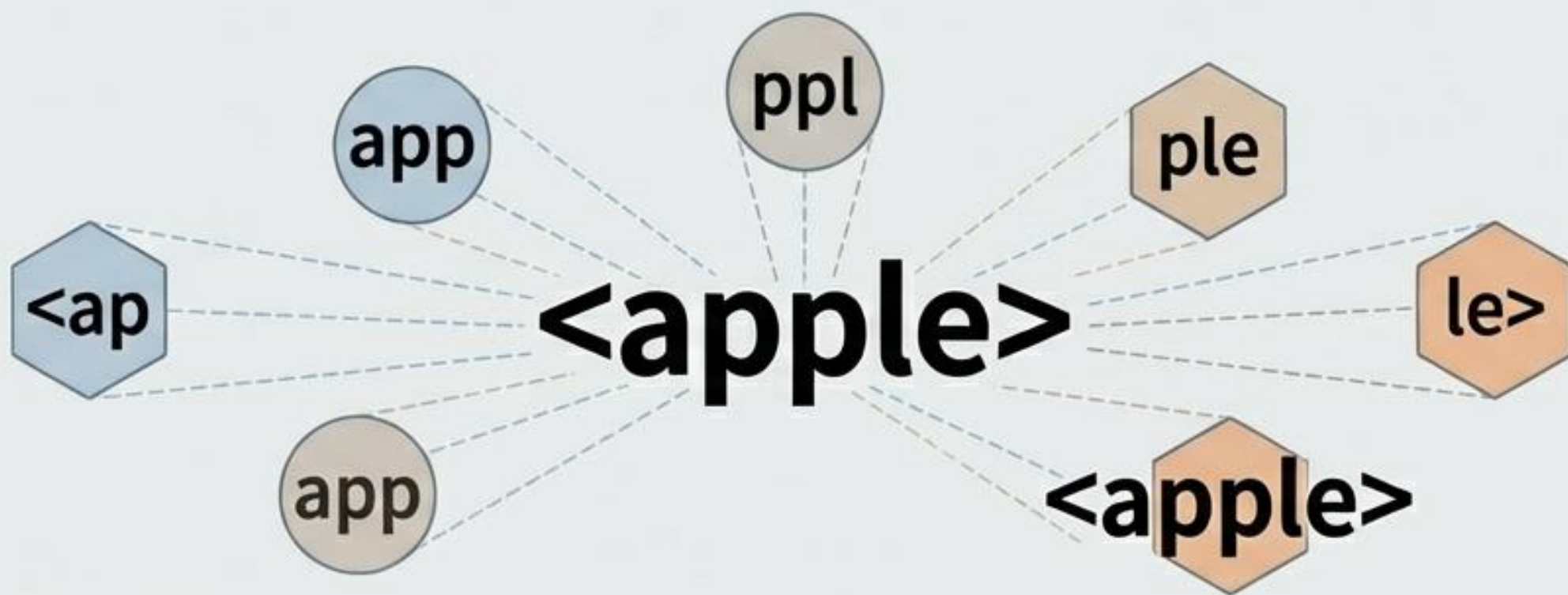
대칭성 보장을 위해 방해항을 편향(Bias)으로 대체

최종 손실 함수 (가중치 함수 $f(X_{ij})$ 적용)

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T w_j + b_i + b_j - \log(X_{ij}))^2$$

단어 내부로의 현미경 탐험: 패스트텍스트(FastText)

- 단어를 쪼갤 수 없는 단위가 아닌, 글자 단위 n-gram(Subword)의 집합으로 취급
- 최종 단어 벡터 = 내부 단어 토큰 벡터들의 총합



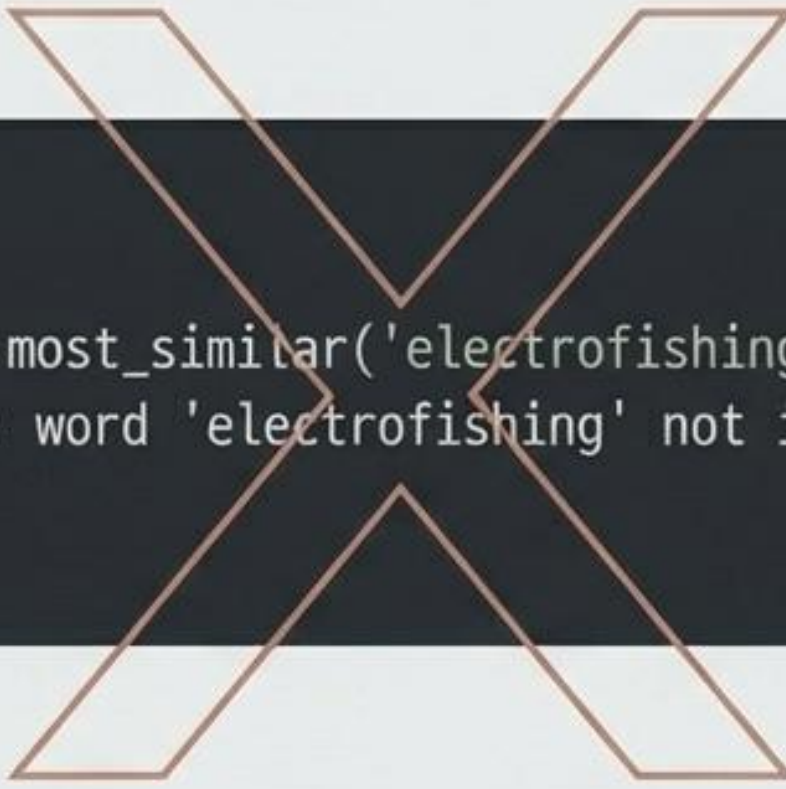
- ‘자연어처리’ 분해:

자 _  ㅇ 여 닌 _  ㅇ 어 _  처 처 _  리 | _

초성/중성/종성 분리로 오타 및 노이즈에 극강의 방어력 확보

OOV(미등록 단어)와 희귀 단어를 극복하다

Word2Vec의 한계



```
model.wv.most_similar('electrofishing')  
KeyError: word 'electrofishing' not in vocabulary
```

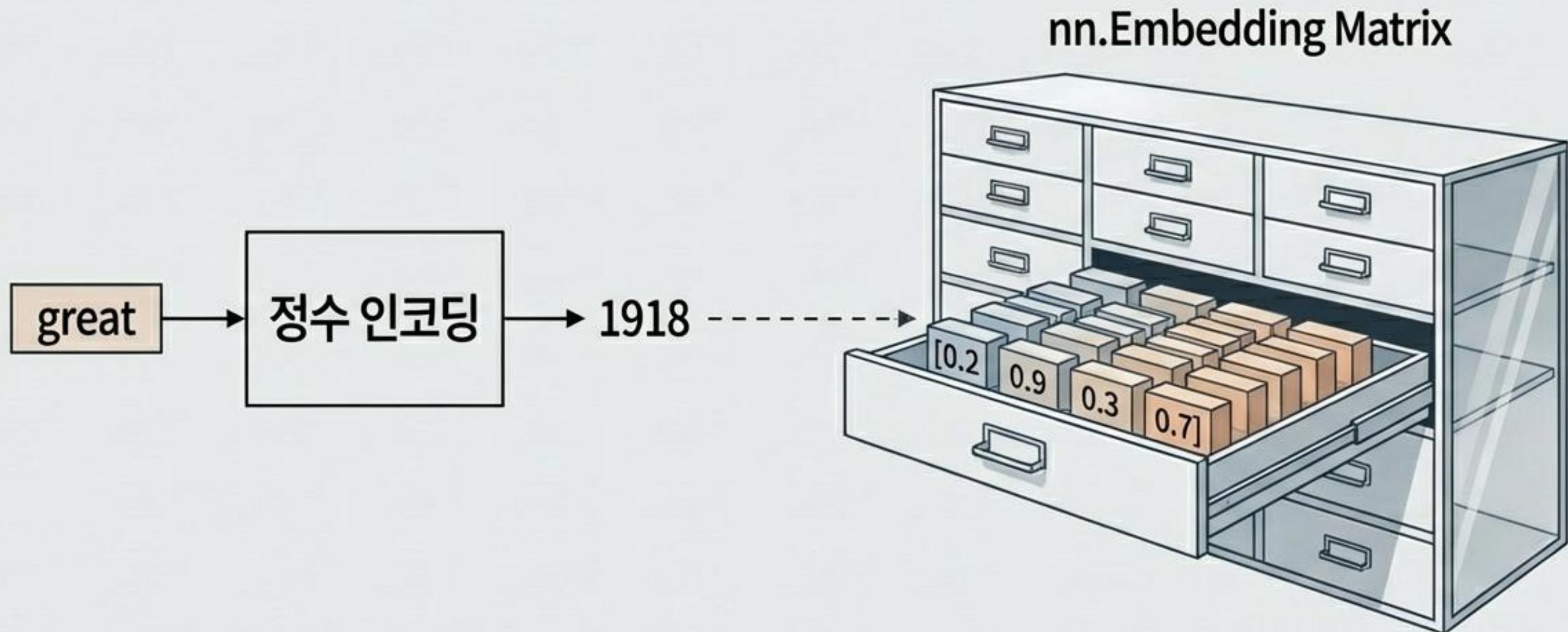
FastText의 해결

```
model.wv.most_similar('electrofishing')  
Output:  
( 'electrolux', 0.793)  
( 'electrolyte', 0.782)  
( 'electro', 0.779)
```

내부 단어(electro, fishing)가 학습되어
있다면 미등록 단어도 벡터 추론 가능

파이토치 임베딩 층의 본질: 룩업 테이블(Lookup Table)

- 입력 정수 인덱스에 매핑되는 밀집 벡터를 테이블에서 그대로 복사해오는 작업
- 인공 신경망 학습 과정(역전파)에서 이 테이블의 가중치 행렬이 지속적으로 업데이트 됨



[실습] PyTorch nn.Embedding() 구현

단어 집합의 총 크기

```
import torch.nn as nn
```

```
embedding_layer = nn.Embedding(  
    num_embeddings=8,  
    embedding_dim=3,  
    padding_idx=1  
)
```

출력할 밀집 벡터의 차원 수

패딩 토큰은 0 벡터로 고정하여 학습 배제

```
print(embedding_layer.weight)
```

```
Parameter containing:
```

```
tensor([[ -0.1778, -1.9974, -1.2478], # <unk> (idx 0)  
        [ 0.0000,  0.0000,  0.0000], # <pad> (idx 1, 고정)  
        [ 1.0921,  0.0416, -0.7896], # code  
        ...  
        [-0.4817, -1.1222,  2.2774]], requires_grad=True)
```

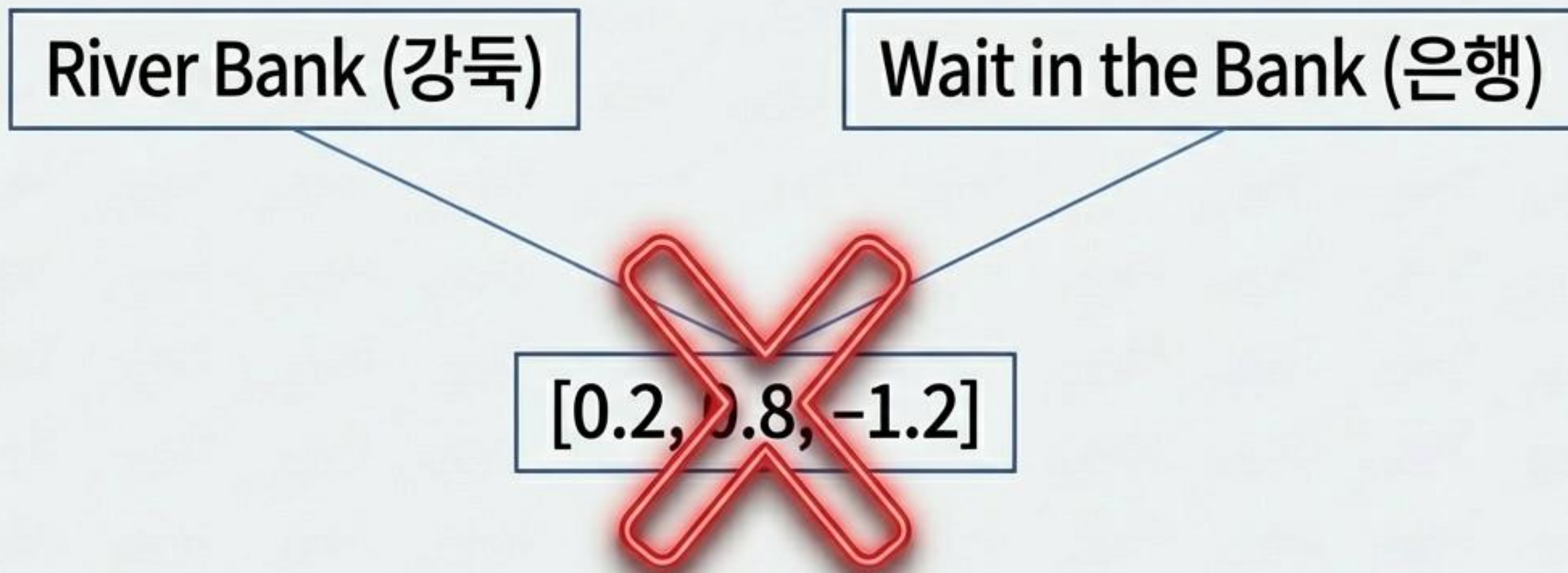

거인의 어깨 위에 서다: 사전 훈련된 임베딩 로드

데이터가 부족한 도메인에서 밑바닥부터 학습하는 한계를 극복



```
import gensim
word2vec_model = gensim.models.KeyedVectors.load_word2vec_format('GoogleNews.bin')
print(word2vec_model.similarity('this', 'is')) # 0.407
print(word2vec_model.similarity('post', 'book')) # 0.057
print(word2vec_model['book'].shape) # (300,)
```

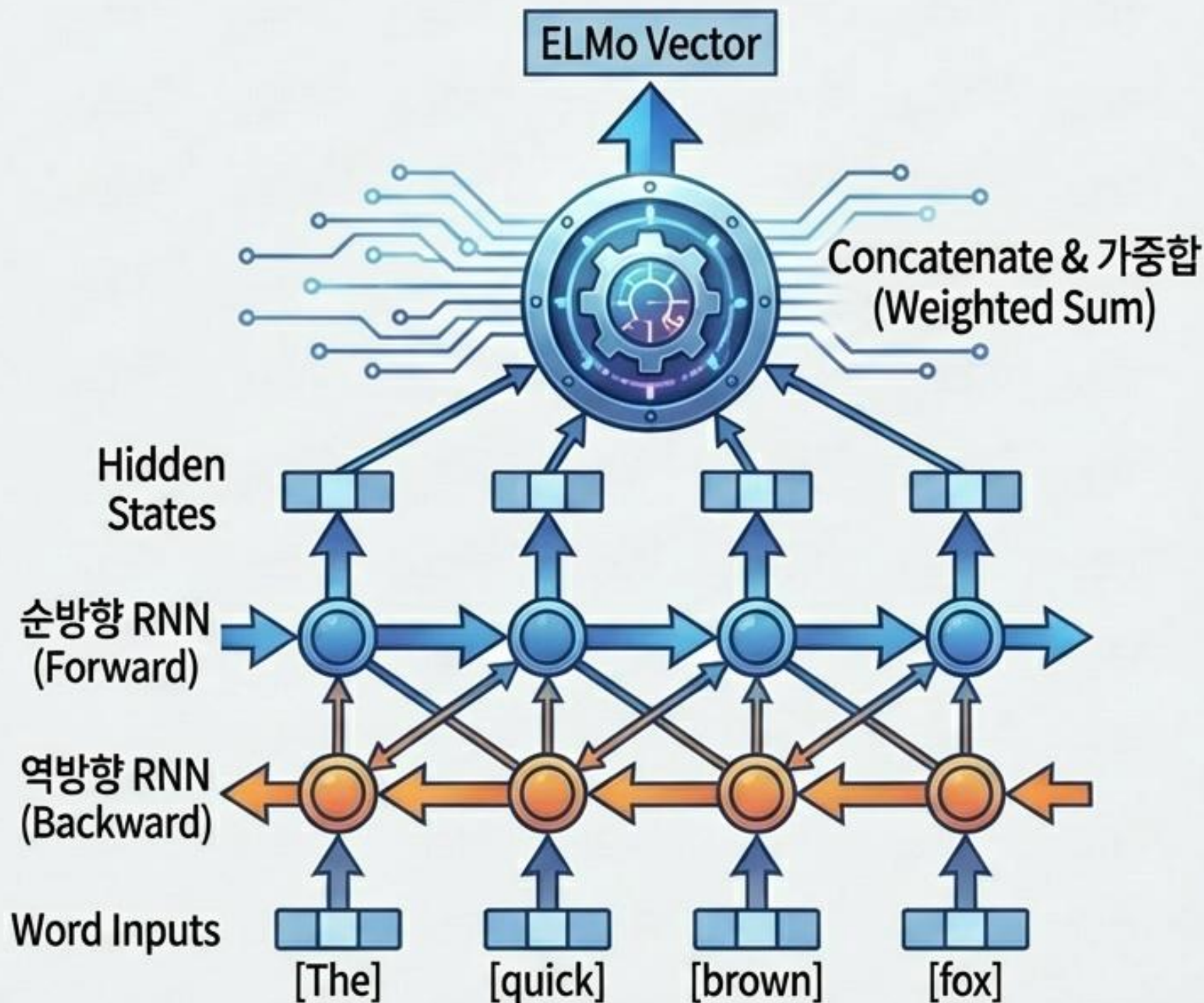

문맥의 부재를 넘어: Contextualized Word Embedding



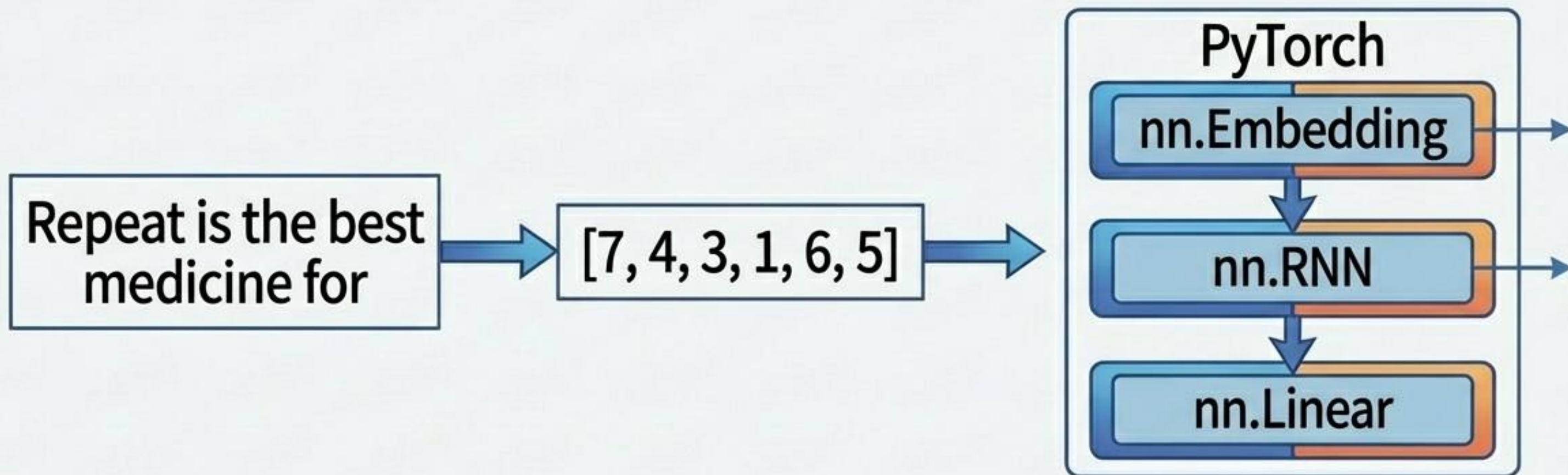
- Word2Vec과 GloVe의 치명적 한계: 다의어 처리 불가
- 문맥(Context)에 따라 카멜레온처럼 변하는 임베딩의 필요성 대두
- 2018년 ELMo (Embeddings from Language Model)의 등장

ELMo의 핵심: 양방향 언어 모델(biLM)과 가중합(Weighted Sum)

- biLM 학습: 순방향과 역방향 RNN을 각각 독립적으로 학습
- 각 층의 은닉 상태를 연결하고, 학습 가능한 가중치를 곱하여 최종 벡터 도출
- 기존 GloVe 벡터와 결합하여 NLP 태스크 성능 극대화



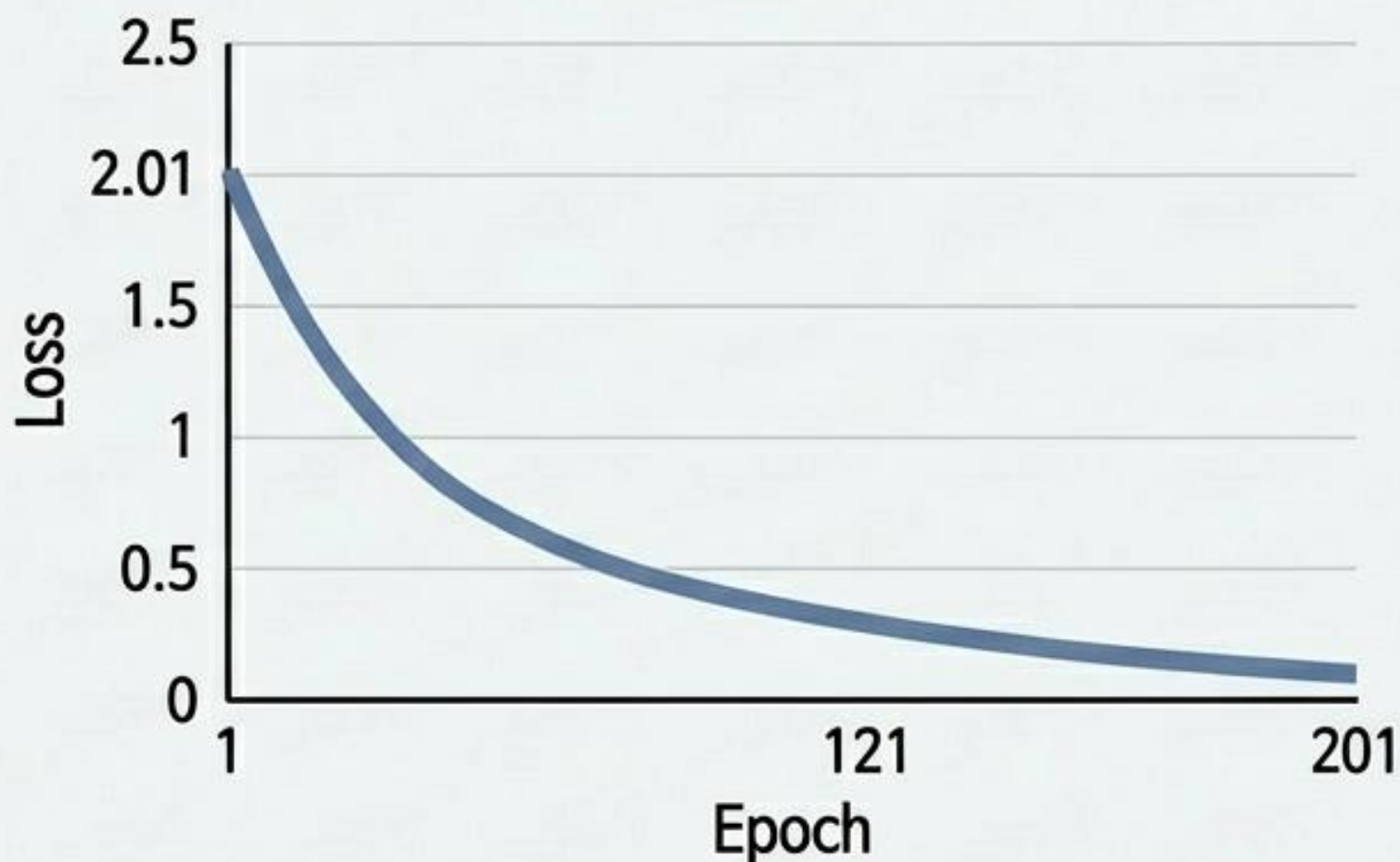
[최종 실습] 단어 단위 RNN 모델 구축 (임베딩 층 적용)



```
class Net(nn.Module):  
    def __init__(self, vocab_size, input_size, hidden_size):  
        super(Net, self).__init__()  
        self.embedding_layer = nn.Embedding(vocab_size, input_size)  
        self.rnn_layer = nn.RNN(input_size, hidden_size, batch_first=True)  
        self.linear = nn.Linear(hidden_size, vocab_size)
```


학습의 완성: 언어 모델의 다음 단어 예측

학습 곡선: 손실 감소 (Loss Reduction)



훈련 로그 요약:

[01/201] Loss: 2.0184

결과: Repeat the the the the medicine best

[121/201] Loss: 0.2992

결과: Repeat is the best medicine for memory

[201/201] Loss: 0.0964

결과: Repeat is the best medicine for memory

단순한 숫자에 불과했던 기호(원-핫 인코딩)가, 룩업 테이블과 RNN의 은닉 상태를 거쳐 문맥을 이해하는 AI 언어 모델로 진화함.